GLB 3rd Year Assessment 2 - Slot 3

Contest Problem Report

Contest Code	GLB3A2S3
Total Problems	5
Coding Problems	5

Problem Name: Chef Finds Minimum Bit Flips to Convert Number

Problem Statement:

Chef loves working with bits and binary numbers. Today, Chef has two integers: an *initialNumber* and a *targetNumber*. Chef wants to transform the *initialNumber* into the *targetNumber* by flipping bits, that is, changing a bit from *0* to *1* or from *1* to *0*. Each flip changes exactly one bit in the binary representation of the number. Chef wants to do this transformation using the minimum number of bit flips possible.

Help Chef find out the least number of bit flips required to convert the *initialNumber* into the *targetNumber*.

Function Declaration

- **Function Name**
  - *calculateMinimumBitFlips*
- **Parameters**
  - *initialNumber*: The starting integer value.
  - *targetNumber*: The target integer value to reach by flipping bits.

Both integers are non-negative and fit within the standard 32-bit integer range.

- **Return Value**
  - Returns an integer representing the minimum count of bit positions that must be flipped.
  - The output is a non-negative integer indicating the bit difference between *initialNumber* and *targetNumber*.

Constraints

- $1 \leq T \leq 10^5$
- $0 \leq initialNumber, targetNumber \leq 10^9$

Input Format

- The first line contains a single integer *T*, the number of test cases.
- Each of the next *T* lines contains two space-separated integers: *initialNumber* and *targetNumber*.

Output Format

- For each test case, print a single line containing the minimum number of bit flips Chef must perform to convert *initialNumber* into *targetNumber*.

Sample Input 1:

```
3
10 20
7 15
5 5
```

Sample Output 1:

```
4
1
0
```

Explanation:

For the first test case:*10* in binary is *1010* and *20* in binary is *10100*.
If we align them as *01010* and *10100*, they differ at 4 bit positions.
So, the answer is *4*.

For the second test case:*7* in binary is *0111* and *15* in binary is *1111*.
They differ at only 1 bit position (the most significant bit).
So, the answer is *1*.

For the third test case:*5* and *5* are the same number, so no bits need to be flipped.
So, the answer is *0*.

Problem Name: Check Algorithm

Problem Statement:

One day, Saeed was teaching a string compression algorithm. This algorithm finds all maximal substrings which contains only one character repeated one or more times (a substring is maximal if it we cannot add one character to its left or right without breaking this property) and replaces each such substring by the string "cK", where K is the length of the substring and c is the only character it contains. For example, "aabaaa" is compressed to "a2b1a3".

Saeed wanted to check if the students understood the algorithm, so he wrote a string S on the board and asked the students if the algorithm is effective on S, i.e. if the string created by compressing S is strictly shorter than S. Help them answer this question.

Input

- The first line of the input contains a single integer T denoting the number of test cases. The description of T test cases follows.
- The first and only line of each test case contains a single string S.

Output

For each test case, print a single line containing the string `"YES"` if the algorithm is effective on S or `"NO"` if it is not.

Constraints

- $1 \le T \le 100$
- $1 \le |S| \le 10^3$
- S may consist of only lowercase English letters.

Subtasks

Subtask #1 (100 points): original constraints

Sample Input 1:

3 bbbbbbbbbbaa c aaaaaaaaaabcdefgh

Sample Output 1:

YES NO NO

Explanation:

Example case 1:

- The compressed string of "bbbbbbbbbbaa" is "b10a2", which is shorter.
- The compressed string of "c" is "c1", which is not shorter than "c".
- The compressed string of "aaaaaaaaaabcdefgh" is "a10b1c1d1e1f1g1h1", which is not shorter than "aaaaaaaaaabcdefgh" (both strings have length 17).

Problem Name: Smallest Numbers of Notes

Problem Statement:

Consider a currency system in which there are notes of six denominations, namely, Rs. 1, Rs. 2, Rs. 5, Rs. 10, Rs. 50, Rs. 100.
If the sum of Rs. **N** is input, write a program to compute smallest number of notes that will combine to give Rs. **N**.

Input

The first line contains an integer **T**, total number of testcases. Then follow **T** lines, each line contains an integer **N**.

Output

For each test case, display the smallest number of notes that will combine to give **N**, in a new line.

Constraints

- 1 **≤T≤** 1000
- 1 **≤N≤** 1000000

Sample Input 1:

3 1200 500 242

Sample Output 1:

12 5 7

Explanation:

Problem Name: Find Pairs with Given Sum in Doubly Linked List

Problem Statement:

You are given a **sorted doubly linked list** of **distinct positive integers** and an integer **target**. Your task is to **find all pairs** `(a, b)` such that:

a + b == target and *a < b*

Return or print all such pairs in ascending order of a.\ If no pairs exist, print an empty list `[]`.

Function Declaration

Function Name

findPairs – This function finds all pairs of values in a **sorted doubly linked list** such that:

- The sum of the pair equals the given target
- The pair satisfies *a + b = __TEXT_PLACEHOLDER0* and *a < b*

Parameters

- *head* : A pointer to the head (first node) of the doubly linked list.
- *tail* : A pointer to the tail (last node) of the doubly linked list.
- *target* : The integer sum for which valid pairs must be found.

Return Value

- This function does not return a value.
- It prints all valid pairs in the format:
  `[a, b] [c, d] ...` in increasing order of a.
- If no valid pairs exist, it prints `[]`.

Constraints

- $1 \leq N \leq 10^5$
- $1 \leq$ __TEXT_PLACEHOLDER0$\leq 10^5$
- $1 \leq$ __TEXT_PLACEHOLDER0$\leq 10^5$
- The doubly linked list is sorted in **strictly increasing order**.
- All node values are **distinct**.

Input Format

- The first line contains an integer *N* — the number of nodes in the doubly linked list.
- The second line contains *N* space-separated integers representing the node values in sorted order.
- The third line contains an integer *target* — the required sum.

Output Format

- Print all valid pairs whose sum equals `target`, in the format:

`[a, b] [c, d] ...`

- If no such pairs exist, print `[]`.

Sample Input 1:

```
7
1 2 4 5 6 8 9
7
```

Sample Output 1:

```
[1, 6] [2, 5]
```

Explanation:

The pairs that sum up to 7 are `(1,6)` and `(2,5)`.

Sample Input 2:

```
3
1 5 6
6
```

Sample Output 2:

```
[1, 5]
```

Explanation:

Problem Name: Flatten a Linked List

Problem Statement:

Alice has a **strange singly-linked list**. A usual singly linked-list or a *not-strange linked list* consist of nodes pointing to exactly one node (except for the the tail node which points to null). However, in case of a **strange singly-linked list**, instead of pointing towards a single linked-list node, some nodes of the linked-list are pointing towards two nodes! Each node in the strange linked-list has two pointers named :

- **next**
- **child**

The “child” pointer of a node may again point towards a strange singly linked list.

Alice wants to restructure this linked-list into a *not-strange linked list*. While doing so, she must follow the following rules.

Let **cur** be a node in restructured linked-list, then

- **cur.next** must occur after cur in the restructured linked-list.
- All the nodes (if any) which are part of **cur.child** linked list must occur after the node cur and before the node **cur.next** in the restructured linked list.
- Let **cur** be a node in the restructured linked-list, then **cur.child** must be “null”.

NOTE : Refer to sample case for a visual depiction.

Input Format

- The first line of the input contains a single integer T - the number of test cases. The description of T test cases follows.

- The first line of each test case contains a single integer *N*.

- The following *N – 1* lines contain three space separated integers *u, v* and *type*. If the value of *type* is equal to 0, then the next pointer of node with value equal to *u* points towards the node with value equal to *v*. If the value of type is equal to 1, then the child pointer of node with value equal to *u* points towards the node with value equal to *v*.

- For C++ language, you need to:

Complete the function in the submit solution tab:

bool flatten(Node* head){. . . }

Output Format

For each test case, the function you complete should return the restructured linked-list.

Constraints

- $1 \leq T \leq 100$
- $1 \leq N \leq 2*10^5$
- It is guaranteed that the sum of N over all test cases is less than or equal to $3*10^5$.

Subtasks

- 5 points : There is at most one node whose child pointer in NOT null. All the other nodes have their child pointers as null pointer.
- 95 points : **ORIGINAL CONSTRAINTS**

Sample Input 1:

1 9 4 7 1 1 2 0 2 3 0 6 9 0 3 6 0 2 4 1 4 8 0 6 5 1

Sample Output 1:

1 2 4 7 8 3 6 5 9

Explanation:

![Alt text]("a title")

![Alt text]("a title")